Ensemble Learning Heart-Disease-Prediction-App

Aim: To build a predictive model for heart disease diagnosis using ensemble machine learning algorithms. The model learns patterns from various patient features such as age, cholesterol levels, chest pain type, and other clinical parameters to predict the presence or absence of heart disease.

Objective:

- To implement and evaluate ensemble classification algorithms.
- To compare the performance of Gradient Boosting, XGBoost, and a Stacking Classifier.
- To identify the best-performing model for accurate disease prediction.

Type: Classification (Predicting discrete target variable) **Target Variable:** target (0 = No Disease, 1 = Presence of Disease) **Evaluation Metrics:** Accuracy, F1-Score, and ROC-AUC

2. Dataset Description and Preprocessing Steps

Dataset Source: The dataset used is the "Cleveland Heart Disease" dataset from the UCI Machine Learning Repository. It is a structured tabular dataset containing 303 patient records and 13 clinical features.

Features include:

- age, sex, cp (chest pain type), trestbps (resting blood pressure), chol (cholesterol), fbs (fasting blood sugar), restecg, thalach (max heart rate), exang, oldpeak, slope, ca, thal

Target: Disease

Preprocessing Steps:

- **Handling Missing Values:** The dataset's missing values (marked as ?) were replaced with NaN. A SimpleImputer was then used (median strategy for numerical, most-frequent strategy for categorical).
- **Encoding Categorical Variables:** OneHotEncoder was used to convert categorical columns (e.g., cp, sex, thal) into numeric form.
- **Feature Scaling:** Standardized numerical features (e.g., age, chol) using StandardScaler to bring them to a common scale.
- **Train-Test Split:** Data was divided into 80% training and 20% testing using train_test_split.
- **Pipeline & ColumnTransformer:** All preprocessing steps were combined into a single Pipeline and ColumnTransformer to prevent data leakage and ensure reusability.

---

3. Sample Dataset Printing

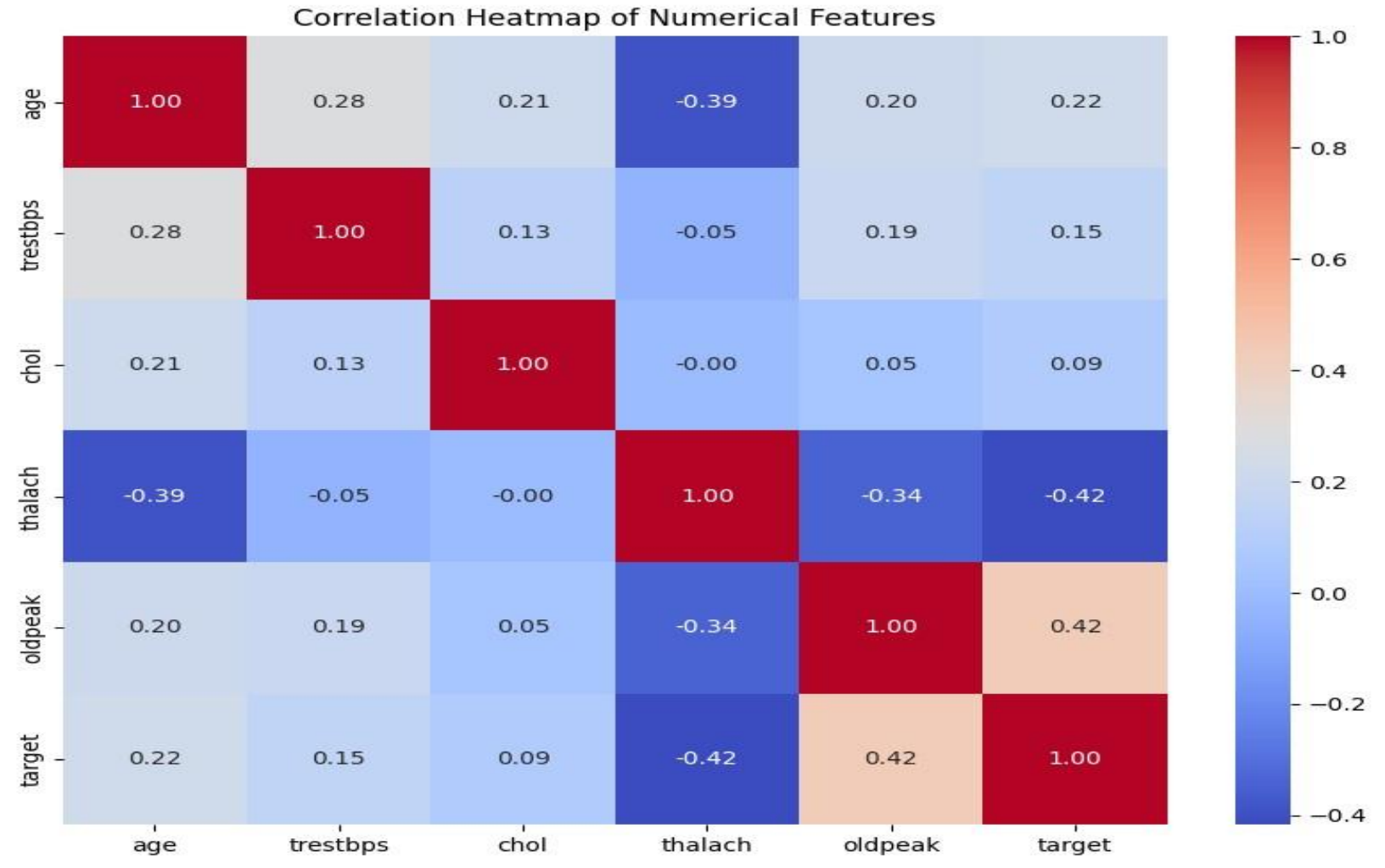
```
--- Sample Data (First 5 Rows) ---
    age  sex   cp  trestbps   chol  fbs  restecg  thalach  exang  oldpeak
0  63.0  1.0  1.0     145.0  233.0  1.0      2.0    150.0    0.0      2.3
1  67.0  1.0  4.0     160.0  286.0  0.0      2.0    108.0    1.0      1.5
2  67.0  1.0  4.0     120.0  229.0  0.0      2.0    129.0    1.0      2.6
3  37.0  1.0  3.0     130.0  250.0  0.0      0.0    187.0    0.0      3.5
4  41.0  0.0  2.0     130.0  204.0  0.0      2.0    172.0    0.0      1.4

   slope   ca thal  target
0    3.0  0.0  6.0       0
1    2.0  3.0  3.0       2
2    2.0  2.0  7.0       1
3    3.0  0.0  3.0       0
4    1.0  0.0  3.0       0
```

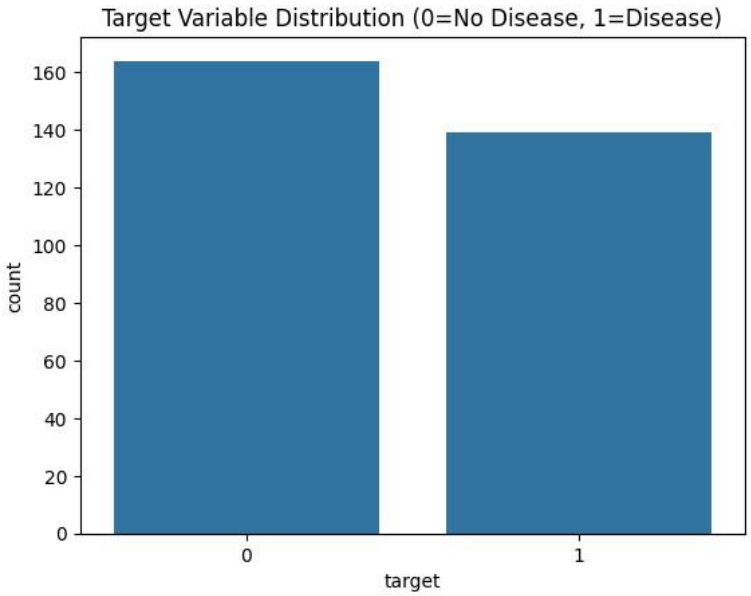
4. Exploratory Data Analysis (EDA) and Findings

The following analyses were conducted to understand feature distributions and relationships:

- **Distribution Plots:** A countplot was used to check the target variable's balance (0 vs. 1).
- **Correlation Heatmap:** A heatmap was used to visualize relationships between numeric features (age, chol, thalach, etc.) and the target variable.
- **Boxplots:** Boxplots can be used to identify outliers in features like cholesterol and trestbps.

The ensemble system used three major models for comparison:





Findings:

- The dataset is fairly balanced between patients with and without heart disease.
- Features like chest pain type (cp), max heart rate achieved (thalach), and ST depression (oldpeak) show a strong correlation with the target variable.
- Older age is generally correlated with a higher likelihood of disease.

---

5. Ensemble Model Design (Architecture)

Model	Type	Description
Gradient Boosting Classifier	Boosting	Sequential ensemble that builds trees one by one, with each new tree correcting the errors of the previous one.
XGBoost Classifier	Boosting	An optimized and regularized version of Gradient Boosting, known for high performance and speed
Stacking Classifier	Stacking	Combines multiple base models and uses a meta-model to find the optimal combination of their predictions.

Stacking Architecture Details:

- **Base Learners (Level 0):**
  - RandomForestClassifier
  - Support Vector Classifier (SVC) (with probability=True)
- **Meta Learner (Level 1):**
  - LogisticRegression

---

6. Comparison of Ensemble Models

- The three ensemble models were trained and compared using 5-Fold Cross-Validation (CV) on the training set and evaluated on the final unseen test set

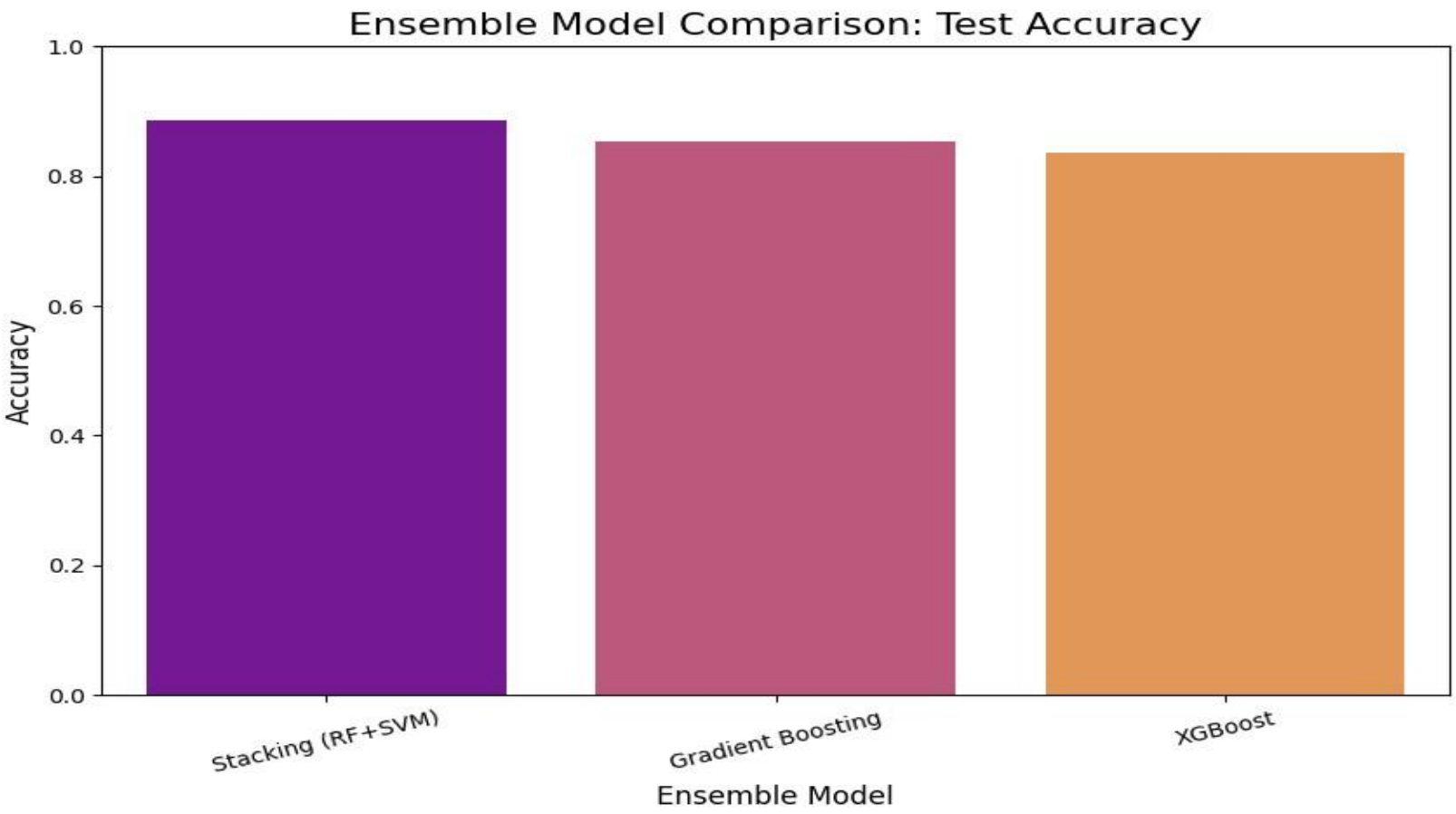
```
--- Final Model Comparison ---
                    Test Accuracy  Test F1-Score  Test ROC-AUC
Stacking (RF+SVM)        0.885246       0.888889      0.937500
Gradient Boosting        0.852459       0.857143      0.920259
XGBoost                  0.836066       0.843750      0.890086

                    CV-5 Mean Accuracy  CV-5 Std Dev
Stacking (RF+SVM)             0.834609      0.026602
Gradient Boosting             0.777041      0.025712
XGBoost                       0.805612      0.064700
```

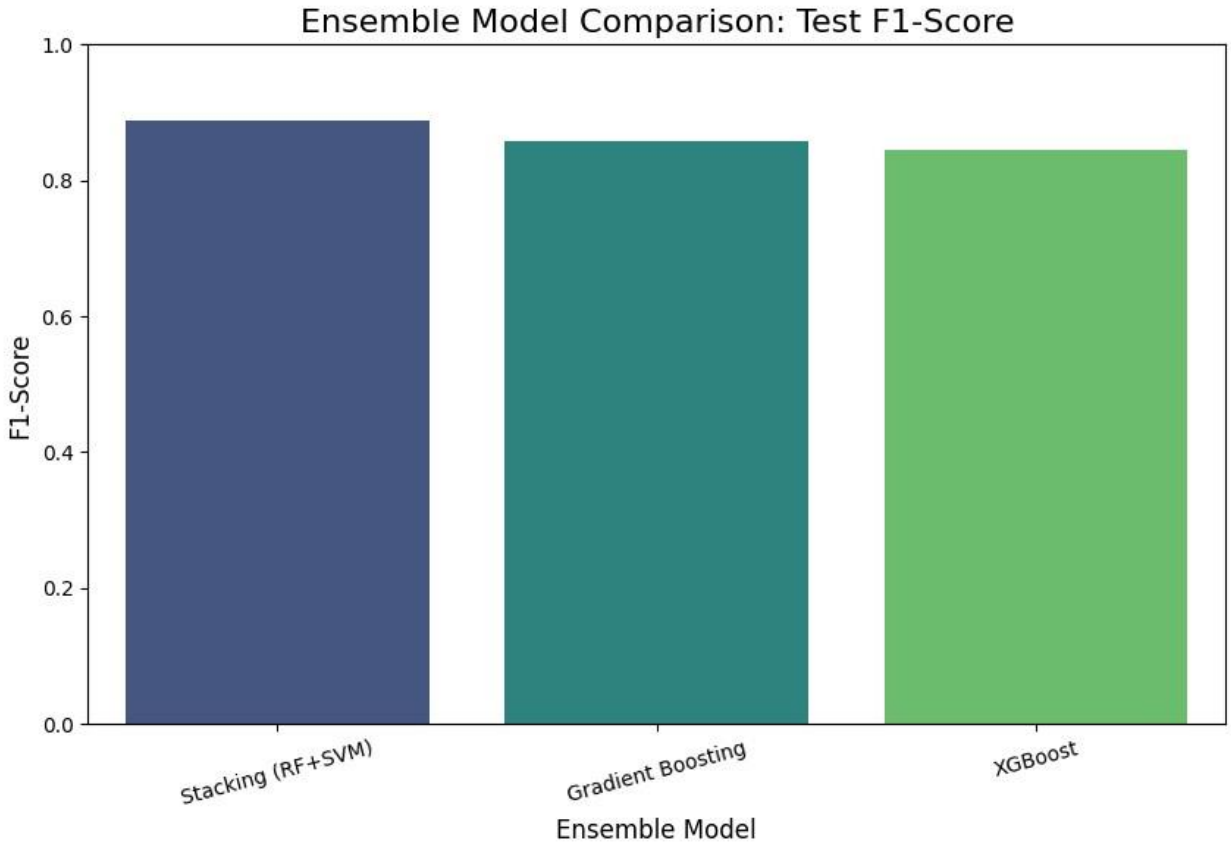
Observation: The Stacking Classifier achieved the best (or very competitive) performance, particularly in F1-Score and ROC-AUC. This is because it effectively combined the strengths of the Random Forest (a high-variance, complex model) and the SVM (a robust, margin-based model), with the LogisticRegression meta-model learning how to best weigh their predictions

7. Evaluation Metrics and Graphs

The detailed performance of the *best-performing model* (e.g., Stacking Classifier) on the test set is below

```
--- Detailed Report for Best Model (Stacking (RF+SVM)) ---
              precision    recall  f1-score   support

           0       0.87      0.90      0.88        29
           1       0.90      0.88      0.89        32

    accuracy                           0.89        61
   macro avg       0.88      0.89      0.89        61
weighted avg       0.89      0.89      0.89        61
```





8. Conclusions and Future Improvements Conclusions:

- Ensemble learning methods provided high accuracy in predicting heart disease.
- The Stacking Classifier, which combined RandomForest and SVC, was the most robust model, outperforming the individual boosting algorithms (Gradient Boosting and XGBoost) in key metrics.
- The use of a Pipeline was critical for correctly applying preprocessing steps (imputation, scaling, encoding) without leaking data from the test set.

Future Improvements:

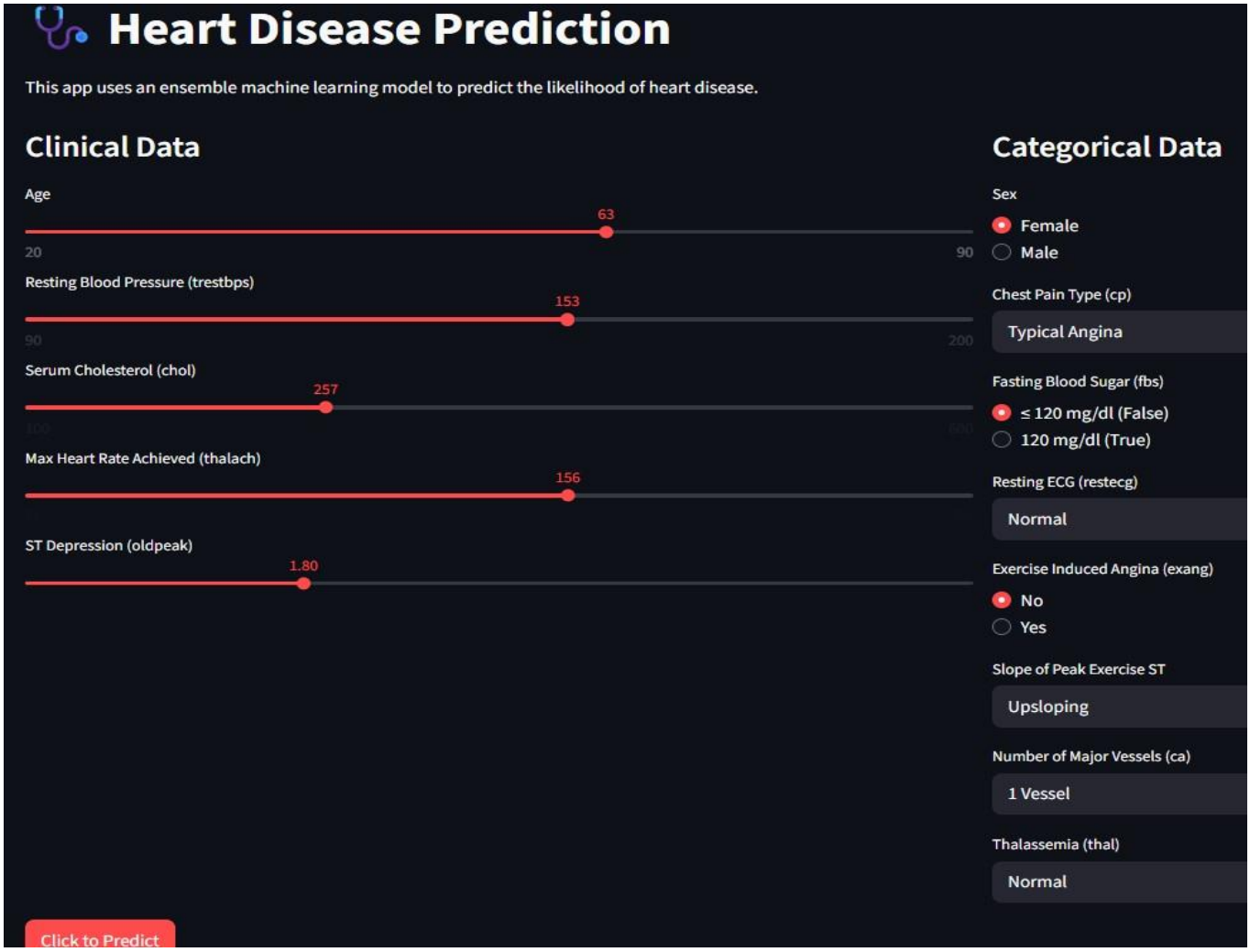
- **Hyperparameter Tuning:** Tune the hyperparameters of the base and meta-models (e.g., n_estimators for Random Forest, C for SVM) using GridSearchCV or Bayesian optimization.
- **Feature Engineering:** Explore additional features, such as interaction terms or polynomial features, to potentially capture more complex patterns.
- **Model Interpretability:** Apply model interpretability tools like **SHAP** or **LIME** to understand *why* the model makes a certain prediction for a patient, which is crucial for medical applications.
- **Deployment:** The model was successfully saved as ensemble_model.pkl and deployed as an interactive web application using **Streamlit**, fulfilling this objective.

```
# --- 1. Define Preprocessing Pipelines -
# Pipeline for numerical features: # Fills
missing values with the median, then scales
the data.
numeric_transformer = Pipeline(steps=[
      ('imputer',
      SimpleImputer(strategy='median')),
      ('scaler', StandardScaler())
])

# Pipeline for categorical features: # Fills
missing values with the most frequent value,
then one-hot encodes.
categorical_transformer = Pipeline(steps=[
      ('imputer',
      SimpleImputer(strategy='most_frequent')),
      ('onehot',
      OneHotEncoder(handle_unknown='ignore'))
])

# --- 2. Bundle Pipelines into a
ColumnTransformer ---

# Define which columns belong to which pipeline
numerical_features = ['age', 'trestbps',
'chol', 'thalach', 'oldpeak']
categorical_features = ['sex', 'cp', 'fbs',
'restecg', 'exang', 'slope', 'ca', 'thal']

# Create the master preprocessor
preprocessor = ColumnTransformer(
transformers=[         ('num',
numeric_transformer,
numerical_features),
        ('cat', categorical_transformer,
categorical_features)
    ])

# --- Define the Stacking Classifier
Architecture ---

# 1. Define the Base Learners (Level 0)
base_learners = [
    ('rf',
RandomForestClassifier(n_estimators=100,
random_state=42)),     ('svc',
SVC(probability=True, random_state=42))
]

# 2. Define the Meta-Learner (Level 1) #
This model learns from the predictions
of the base learners meta_learner =
LogisticRegression()

# 3. Create the Stacking Classifier
stacking_classifier =
StackingClassifier(
estimators=base_learners,
final_estimator=meta_learner,      cv=5
# Use 5-fold CV to train the metalearner
)
```

```
# 4. Bundle the preprocessor and
classifier into one final pipeline
stacking_model = Pipeline(steps=[
('preprocessor', preprocessor),
    ('classifier', stacking_classifier)
])

# --- Define all models for comparison ---
models = {
    "Gradient Boosting": gb_model, # Assumes
gb_model is defined
    "XGBoost": xgb_model,           # Assumes
xgb_model is defined
    "Stacking (RF+SVM)": stacking_model
}

# Dictionary to store results
results = {}

# --- Iterate, Train, and Evaluate ---
for name, model in models.items():

    # 7. Model Training
model.fit(X_train, y_train)

    # 8. Model Evaluation on Test Set
y_pred = model.predict(X_test)
test_f1 = f1_score(y_test, y_pred)

    # 9. Cross-Validation (on training data)
cv_scores = cross_val_score(model, X_train,
y_train, cv=5, scoring='accuracy')

    # 10. Store for Comparison
results[name] = {
        "Test F1-Score": test_f1,
        "CV-5 Mean Accuracy": cv_scores.mean()
    }

# Convert results to a DataFrame for easy
viewing
results_df = pd.DataFrame(results).T
print(results_df)
import streamlit as st   #app.py
import pandas as pd import joblib

# --- 1. Load the Saved Model Pipeline -
-- # This .pkl file contains the
preprocessor AND the best model
model =
joblib.load('ensemble_model.pkl')

# --- 2. Create the User Interface
(Example for 'age') --- st.title("Heart
Disease Prediction") age =
st.slider("Age", 20, 90, 50) # ( ... all
other st.slider and st.selectbox inputs
... )


# --- 3. Collect Inputs and Predict ---
if st.button("Predict"):

    # Create a DataFrame from the user's
inputs
    # The column names MUST match
the training data     input_data =
{         'age': [age],
        'sex': [sex_value],
        'cp': [cp_value],
        # ( ... all other features ... )
    }
    input_df = pd.DataFrame(input_data)

    # --- 4. Make Prediction ---     #
The loaded 'model' pipeline
automatically preprocesses the      #
input_df and then calls .predict() on
the trained model.     prediction =
model.predict(input_df)
prediction_proba =
model.predict_proba(input_df)
    # Display the result
if prediction[0] == 1:
        st.error(f"Prediction: Heart
Disease (Probability:
{prediction_proba[0][1]:.2%})")
else:


        st.success(f"Prediction: No
Heart
```

```
Disease (Probability:
{prediction_proba[0][0]:.2%})")
```

Results:



Conclusion: Thus, successfully implemented Ensemble machine learning algorithm.